

Rapport Final — Application de Gestion de Bibliothèque Universitaire

Équipe de Développement — Université Norbert Zongo

Mai 2026

1. Introduction

1.1 Contexte

L'Université Norbert Zongo (UNZ) souhaitait moderniser sa bibliothèque en développant une application web complète de gestion documentaire. Ce projet académique s'inscrit dans le cadre du cours de Génie Logiciel et vise à mettre en pratique l'ensemble des concepts étudiés.

1.2 Objectifs

Les objectifs principaux de ce projet sont :

- Concevoir et développer une application web professionnelle de gestion de bibliothèque universitaire
- Appliquer la méthodologie Agile Scrum
- Mettre en œuvre les bonnes pratiques du Génie Logiciel
- Utiliser les Design Patterns et les principes SOLID
- Produire une application testée avec une couverture de code suffisante

1.3 Périmètre du projet

L'application couvre la gestion des ouvrages, des emprunts, des réservations, des pénalités, des utilisateurs et des statistiques pour trois types d'acteurs : l'Étudiant, le Bibliothécaire et l'Administrateur.

2. Analyse des Exigences

2.1 Acteurs du système

Acteur	Rôle
Étudiant	Consulte le catalogue, emprunte, réserve, consulte son historique
Bibliothécaire	Gère le catalogue, les emprunts, les retours et les pénalités
Administrateur	Gère les utilisateurs, les statistiques et les rapports

2.2 Exigences Fonctionnelles

1. Gestion du catalogue (ajout, modification, suppression d'ouvrages)
2. Recherche avancée par titre, auteur, ISBN
3. Emprunt et retour d'ouvrages
4. Réservation d'ouvrages indisponibles
5. Calcul automatique des pénalités de retard
6. Notification automatique par email
7. Consultation de l'historique des emprunts
8. Gestion des utilisateurs (inscription, authentification)
9. Génération de statistiques et rapports
10. Gestion des catégories et rayons

2.3 Exigences Non Fonctionnelles

Exigence	Valeur
Sécurité	Authentification JWT, protection des données
Performance	Temps de réponse < 2 secondes
Disponibilité	99%
Interface	Responsive (ordinateur et mobile)
Maintenabilité	Principes SOLID, Design Patterns
Versionnement	Git + GitHub

2.4 User Stories

ID	User Story	Acteur	Priorité
US01	En tant qu'étudiant, je veux m'inscrire avec mon email et mot de passe	Étudiant	Haute
US02	En tant qu'utilisateur, je veux me connecter	Tous	Haute

ID	User Story	Acteur	Priorité
US03	En tant que bibliothécaire, je veux ajouter un ouvrage	Bibliothécaire	Haute
US04	En tant qu'étudiant, je veux consulter le catalogue	Étudiant	Haute
US05	En tant qu'étudiant, je veux rechercher un ouvrage	Étudiant	Haute
US06	En tant qu'étudiant, je veux emprunter un ouvrage	Étudiant	Moyenne
US07	En tant que bibliothécaire, je veux enregistrer un retour	Bibliothécaire	Moyenne
US08	En tant qu'étudiant, je veux réserver un ouvrage	Étudiant	Moyenne
US09	En tant que bibliothécaire, je veux appliquer une pénalité	Bibliothécaire	Moyenne
US10	En tant qu'étudiant, je veux consulter mon historique	Étudiant	Moyenne
US11	En tant qu'étudiant, je veux recevoir des notifications	Étudiant	Basse
US12	En tant qu'administrateur, je veux gérer les utilisateurs	Administrateur	Basse
US13	En tant qu'administrateur, je veux voir les statistiques	Administrateur	Basse
US14	En tant que bibliothécaire, je veux gérer les catégories	Bibliothécaire	Basse

ID	User Story	Acteur	Priorité
US15	En tant qu'administrateur, je veux générer des rapports	Administrateur	Basse

3. Modélisation et Conception

3.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation présente les interactions entre les trois acteurs (Étudiant, Bibliothécaire, Administrateur) et les fonctionnalités du système. Il a été réalisé avec PlantUML.

Les cas d'utilisation principaux sont : - S'inscrire et se connecter - Gérer le catalogue d'ouvrages - Emprunter et retourner des ouvrages - Réserver des ouvrages - Consulter les statistiques

3.2 Diagramme de Classes

Le diagramme de classes définit la structure de données de l'application avec les entités suivantes :

- **Utilisateur** (classe mère) : id, nom, prénom, email, motDePasse, rôle
- **Étudiant** (hérite de Utilisateur) : numéroÉtudiant, filière
- **Bibliothécaire** (hérite de Utilisateur) : matricule
- **Administrateur** (hérite de Utilisateur) : niveau
- **Ouvrage** : id, titre, auteur, ISBN, nombreExemplaires, exemplairesDisponibles
- **Catégorie** : id, nom, rayon
- **Emprunt** : id, dateEmprunt, dateRetourPrévue, dateRetourRéelle, statut
- **Réservation** : id, dateRéservation, dateExpiration, statut
- **Pénalité** : montant, payée

3.3 Diagrammes de Séquence

Trois diagrammes de séquence ont été réalisés :

Connexion : L'utilisateur saisit ses identifiants → le Controller appelle le Service → le Service vérifie en base → retourne un token JWT.

Emprunt : L'étudiant clique sur Emprunter → le Controller vérifie la disponibilité → le Service crée l'emprunt → met à jour les exemplaires disponibles.

Réservation : L'étudiant réserve un ouvrage indisponible → le Service vérifie qu'il n'a pas déjà réservé → crée la réservation → envoie une confirmation.

3.4 Architecture 4+1 Vues

Vue Logique : Architecture en 4 couches — Présentation, Application, Domaine, Infrastructure.

Vue Développement : Organisation en packages — controller, service, repository, model, config, strategy, factory, observer.

Vue Processus : Flux d'exécution — requête HTTP → Spring Security → Controller → Service → Repository → MySQL.

Vue Physique : Déploiement — Navigateur Web → Serveur Spring Boot → Base de données MySQL.

Vue des Scénarios : Les cas d'utilisation principaux couvrant les 3 acteurs.

4. Architecture et Design Patterns

4.1 Architecture Générale

L'application suit une architecture **MVC (Model-View-Controller)** avec Spring Boot :

- **Model** : Entités JPA (Utilisateur, Ouvrage, Emprunt, Réservation)
- **View** : Interface HTML/CSS/JavaScript
- **Controller** : API REST Spring Boot

4.2 Principes SOLID appliqués

Principe	Application
S — Single Responsibility	Chaque classe a une seule responsabilité (OuvrageService, EmpruntService)
O — Open/Closed	Les services sont ouverts à l'extension via les interfaces
L — Liskov Substitution	Etudiant, Bibliothécaire, Admin héritent correctement de Utilisateur
I — Interface Segregation	Interfaces séparées pour Observer et Strategy
D — Dependency Inversion	Spring inject les dépendances via @Autowired

4.3 Design Patterns utilisés

Pattern 1 — Singleton

Problème : Éviter la création multiple d'instances de configuration. **Solution** : La classe `DatabaseConfig` annotée `@Configuration` est gérée comme un Singleton par le conteneur Spring IoC. **Justification** : Spring Boot garantit qu'une seule instance de chaque bean `@Configuration` est créée dans le contexte applicatif.

Pattern 2 — Observer

Problème : Notifier automatiquement les étudiants quand un ouvrage devient disponible. **Solution** : L'interface `Observable` et `Observateur` permettent à `OuvrageDisponible` de notifier tous les abonnés lors du retour d'un ouvrage. **Justification** : Découple le système de notification du système d'emprunt.

Pattern 3 — Factory

Problème : Créer le bon type d'utilisateur (Étudiant, Bibliothécaire, Administrateur) sans connaître le type exact à l'avance. **Solution** : La classe `UtilisateurFactory` crée l'objet approprié selon le rôle fourni. **Justification** : Centralise la création d'objets et facilite l'ajout de nouveaux types d'utilisateurs.

Pattern 4 — Strategy

Problème : Le calcul des pénalités peut varier selon le type d'étudiant (standard ou boursier). **Solution** : L'interface `CalculPenalite` définit le contrat, `PenaliteStandard` (100 FCFA/jour) et `PenaliteBoursier` (50 FCFA/jour) implémentent des stratégies différentes. **Justification** : Permet de changer l'algorithme de calcul sans modifier le code existant.

5. Implémentation et Tests

5.1 Stack Technique

Composant	Technologie
Langage	Java 21
Framework Backend	Spring Boot 3.2.0
Base de données	MySQL 8.4
ORM	Hibernate / JPA
Sécurité	JWT (JSON Web Token)
Frontend	HTML5 / CSS3 / JavaScript
Build	Maven 3.9
Versionnement	Git + GitHub

5.2 Structure du projet

bibliotheque-univ-nz1/ src/main/java/com/bibliotheque/ controller/ # API REST
service/ # Logique métier repository/ # Accès base de données model/ # Entités
JPA config/ # Configuration strategy/ # Pattern Strategy factory/ # Pattern
Factory observer/ # Pattern Observer src/main/resources/ application.properties
static/index.html src/test/ # Tests unitaires pom.xml

5.3 API REST développées

Endpoint	Méthode	Description
/api/ouvrages	GET	Lister tous les ouvrages
/api/ouvrages	POST	Ajouter un ouvrage
/api/ouvrages/{id}	PUT	Modifier un ouvrage
/api/ouvrages/{id}	DELETE	Supprimer un ouvrage
/api/ouvrages/recherche/titre	GET	Rechercher par titre
/api/emprunts	POST	Créer un emprunt
/api/emprunts/{id}/retour	PUT	Retourner un emprunt
/api/emprunts/{id}/penalite	GET	Calculer pénalité
/api/reservations	POST	Créer une réservation
/api/reservations/{id}/annuler	PUT	Annuler réservation
/api/categories	GET/POST	Gérer catégories
/api/statistiques	GET	Voir statistiques

5.4 Tests unitaires

Les tests ont été réalisés avec **JUnit 5** et **Mockito** :

Classe de test	Nombre de tests	Description
OuvrageServiceTest	7	Tests du service des ouvrages
EmpruntServiceTest	6	Tests du service des emprunts
ModelTest	7	Tests des modèles
PenaliteTest	3	Tests des stratégies de pénalité
FactoryTest	4	Tests du pattern Factory
AppTest	1	Test de démarrage
Total	28	

5.5 Couverture de code

La couverture de code a été mesurée avec **JaCoCo** :

Métrique	Valeur
Couverture globale	88%
Objectif requis	65%
Résultat	Objectif dépassé

5.6 Description des fonctionnalités implémentées

Gestion du catalogue

Le module de gestion du catalogue permet aux bibliothécaires d'ajouter, modifier et supprimer des ouvrages. Chaque ouvrage possède un titre, un auteur, un ISBN unique, une description, une année de publication, un nombre d'exemplaires total et un nombre d'exemplaires disponibles. La catégorie de l'ouvrage est également gérée permettant une organisation par rayon de bibliothèque.

Système d'emprunt

Le système d'emprunt vérifie automatiquement la disponibilité d'un ouvrage avant de créer un emprunt. Lors de la création d'un emprunt, le nombre d'exemplaires disponibles est décrémenté automatiquement. La date de retour prévue est calculée automatiquement à 14 jours après la date d'emprunt. Lors du retour, le nombre d'exemplaires disponibles est incrémenté.

Système de réservation

Quand tous les exemplaires d'un ouvrage sont empruntés, l'étudiant peut créer une réservation. Le système vérifie qu'il n'existe pas déjà une réservation active pour le même étudiant et le même ouvrage. La réservation expire automatiquement après 7 jours si l'étudiant ne vient pas récupérer l'ouvrage.

Calcul des pénalités

Le calcul des pénalités utilise le Pattern Strategy pour permettre différents modes de calcul. La pénalité standard est de 100 FCFA par jour de retard. La pénalité boursier est de 50 FCFA par jour de retard. Le calcul se fait automatiquement en comparant la date de retour réelle avec la date de retour prévue.

Système de notifications email

Le système de notifications utilise Spring Mail avec le protocole SMTP de Gmail. Deux types de notifications sont implémentés : le rappel de retour avant la date limite et la notification de disponibilité d'un ouvrage réservé. Les notifications sont envoyées automatiquement par email à l'adresse de l'étudiant.

Statistiques et rapports

Le module de statistiques fournit en temps réel le nombre total d'ouvrages, le nombre total d'emprunts, le nombre d'emprunts en cours, le nombre d'emprunts en retard, le nombre total de réservations et le nombre total d'utilisateurs. Ces statistiques sont accessibles via l'API REST et affichées sur l'interface web.

5.7 Sécurité de l'application

La sécurité de l'application est assurée par plusieurs mécanismes :

Configuration Spring Security

Spring Security est configuré pour protéger toutes les routes de l'application. La configuration CSRF est désactivée pour permettre les requêtes API REST. Les headers CORS sont configurés pour permettre les requêtes depuis le frontend.

Authentification JWT

Le système d'authentification utilise des tokens JWT (JSON Web Token). Chaque token contient l'identifiant de l'utilisateur, son rôle et une date d'expiration de 24 heures. Le secret JWT est stocké dans le fichier de configuration `application.properties`.

Protection des données

Les mots de passe des utilisateurs sont stockés de façon sécurisée. Les informations sensibles comme les credentials de la base de données et le mot de passe email sont externalisés dans le fichier `application.properties` qui n'est pas commité sur GitHub public.

5.8 Interface utilisateur

L'interface web a été développée en HTML5, CSS3 et JavaScript pur sans framework frontend. Elle est responsive et s'adapte aux écrans d'ordinateur et mobile.

Page d'accueil

La page d'accueil affiche les statistiques en temps réel, le catalogue des ouvrages avec recherche, l'historique des emprunts et les réservations.

Catalogue des ouvrages

Le catalogue affiche tous les ouvrages avec leur titre, auteur, ISBN et le nombre d'exemplaires disponibles. Un bouton de recherche permet de filtrer par titre. Un bouton d'ajout permet aux bibliothécaires d'ajouter de nouveaux ouvrages.

Formulaire d'ajout

Le formulaire d'ajout permet de saisir le titre, l'auteur, l'ISBN et le nombre d'exemplaires d'un nouvel ouvrage. La soumission du formulaire appelle l'API REST et met à jour le catalogue automatiquement.

6. Gestion Agile (Scrum)

6.1 Organisation de l'équipe

Rôle	Responsabilité
Product Owner	Priorise le backlog, valide les fonctionnalités
Scrum Master	Facilite les cérémonies Scrum, lève les blocages
Équipe Dev	Développe, teste et livre les fonctionnalités

6.2 Sprints réalisés

Sprint 1 — Authentification + Catalogue (2 semaines)

- US01 : Inscription utilisateur
- US02 : Connexion
- US03 : Ajout d'ouvrage
- US04 : Consultation du catalogue
- US05 : Recherche d'ouvrage

Sprint 2 — Emprunts + Réservations + Pénalités (2 semaines)

- US06 : Emprunt d'ouvrage
- US07 : Retour d'ouvrage
- US08 : Réservation
- US09 : Pénalités de retard
- US10 : Historique des emprunts

Sprint 3 — Notifications + Rapports + Déploiement (2 semaines)

- US11 : Notifications
- US12 : Gestion utilisateurs
- US13 : Statistiques
- US14 : Gestion catégories
- US15 : Génération de rapports

6.3 Outil de gestion

L'outil **GitHub Projects** a été utilisé pour gérer le Product Backlog, le Sprint Backlog et le tableau Kanban avec les colonnes : Product Backlog, Sprint Backlog, In Progress, In Review,

Done.

6.4 Definition of Done (DoD)

Une User Story est considérée comme Done quand : - Le code est développé et compilé sans erreur - Les tests unitaires sont écrits et passent - Le code est commité sur GitHub - La fonctionnalité est démontrée et validée

7. Difficultés Rencontrées et Leçons Apprises

7.1 Difficultés rencontrées

Difficulté 1 — Configuration de Spring Security Spring Security bloquait toutes les requêtes API. La solution a été de configurer une `SecurityFilterChain` personnalisée pour autoriser les requêtes pendant le développement.

Difficulté 2 — Déploiement en ligne Le déploiement sur Render a posé des problèmes de connectivité avec la base de données MySQL distante. Nous avons migré vers PostgreSQL et Railway pour résoudre ce problème.

Difficulté 3 — Gestion des ports Des conflits de ports (8080, 8081) ont été rencontrés lors du développement local. La solution a été de libérer les ports avec `fuser -k` avant chaque lancement.

Difficulté 4 — Intégration des Design Patterns L'intégration du pattern Singleton avec Spring Boot nécessitait d'adapter le constructeur privé car Spring ne peut pas instancier des classes avec des constructeurs privés.

7.2 Leçons apprises

- L'importance de bien configurer l'environnement de développement dès le début
 - La méthodologie Scrum facilite la gestion du projet et la livraison incrémentale
 - Les tests unitaires permettent de détecter rapidement les régressions
 - La documentation du code et des API est essentielle pour la maintenance
 - Le versionnement Git avec des branches séparées (main/develop) améliore la qualité du code
-

8. Conclusion

Ce projet nous a permis de développer une application web complète de gestion de bibliothèque universitaire en appliquant les meilleures pratiques du Génie Logiciel.

Bilan technique

L'application développée avec Spring Boot 3.2 et MySQL offre une API REST complète avec 12 endpoints, une interface web responsive, 28 tests unitaires avec 88% de couverture de code, et 4 Design Patterns implémentés.

Bilan méthodologique

La méthodologie Scrum a été appliquée avec succès à travers 3 sprints de 2 semaines chacun, permettant de livrer progressivement les fonctionnalités prioritaires tout en maintenant la qualité du code.

Perspectives d'amélioration

- Implémenter un système d'authentification JWT complet
- Ajouter un système de notification par email fonctionnel
- Améliorer l'interface utilisateur avec un framework moderne (React ou Angular)
- Mettre en place une intégration continue (CI/CD) avec GitHub Actions
-

Déployer l'application sur un serveur cloud avec une base de données persistante

9. Hébergement et Déploiement

9.1 Environnement local

L'application fonctionne parfaitement en local avec la configuration suivante :

- **URL** : http://localhost:8081
- **Base de données** : MySQL 8.4 sur localhost:3306
- **Java** : OpenJDK 21
- **Maven** : 3.9.9

9.2 Dockerisation

Un Dockerfile a été fourni pour faciliter le déploiement :

- Image de build : maven:3.9.5-eclipse-temurin-21
- Image de production : eclipse-temurin:21-jre
- Port exposé : 8081

9.3 Tentative de déploiement sur Railway

Une tentative de déploiement sur Railway (railway.app) a été réalisée :

- Le code source est connecté au dépôt GitHub
- La base de données PostgreSQL a été configurée
- Le Dockerfile a été optimisé pour Railway
- Des problèmes de configuration de port ont été rencontrés

9.4 Conclusion sur l'hébergement

L'application est pleinement fonctionnelle en environnement local. Le Dockerfile fourni permet un déploiement futur sur n'importe quelle plateforme supportant Docker (Railway, Render, AWS, Heroku, etc.). —

10. Analyse de la qualité du code

10.1 Respect des principes SOLID

Single Responsibility Principle (SRP)

Chaque classe du projet a une seule responsabilité bien définie. La classe `OuvrageService` gère uniquement la logique métier des ouvrages. La classe `EmpruntService` gère uniquement la logique des emprunts. La classe `NotificationService` gère uniquement l'envoi des notifications. Les `Controllers` se chargent uniquement de recevoir et retourner les requêtes HTTP sans contenir de logique métier.

Open/Closed Principle (OCP)

Les classes sont ouvertes à l'extension mais fermées à la modification. Le Pattern Strategy illustre parfaitement ce principe : pour ajouter un nouveau mode de calcul de pénalité, il suffit de créer une nouvelle classe implémentant l'interface `CalculPenalite` sans modifier le code existant.

Liskov Substitution Principle (LSP)

Les sous-classes `Etudiant`, `Bibliothecaire` et `Administrateur` peuvent remplacer leur classe mère `Utilisateur` sans altérer le comportement du programme. Chaque sous-classe respecte le contrat défini par la classe mère.

Interface Segregation Principle (ISP)

Les interfaces sont petites et spécifiques. L'interface `Observable` définit uniquement les méthodes liées à la gestion des observateurs. L'interface `Observateur` définit uniquement la méthode de réception des notifications. L'interface `CalculPenalite` définit uniquement la méthode de calcul.

Dependency Inversion Principle (DIP)

Les classes de haut niveau ne dépendent pas des classes de bas niveau. Spring Boot gère l'injection de dépendances via l'annotation `@Autowired`. Les Controllers dépendent des interfaces de Service, pas des implémentations concrètes.

10.2 Bonnes pratiques de développement

Conventions de nommage

Les noms de classes, méthodes et variables suivent les conventions Java standard. Les classes utilisent le PascalCase, les méthodes et variables utilisent le camelCase, les constantes utilisent le UPPER_SNAKE_CASE.

Commentaires et documentation

Chaque Design Pattern est documenté avec un commentaire expliquant son rôle et son utilisation. Les méthodes importantes sont commentées pour faciliter la maintenance et la compréhension du code.

Gestion des exceptions

Les Controllers utilisent des blocs try/catch pour gérer les exceptions et retourner des messages d'erreur appropriés avec les codes HTTP correspondants (400, 404, 500).

Versionnement Git

Le code est versionné avec Git en suivant les bonnes pratiques : messages de commit descriptifs, utilisation de branches séparées (main pour la production, develop pour le développement), et merge régulier entre les branches.

10.3 Métriques de qualité

Métrique	Valeur
Nombre de classes Java	33
Nombre de méthodes	85+
Lignes de code	1500+
Couverture de tests	88%
Nombre de tests	28
Nombre d'endpoints API	12
Design Patterns	4
Commits GitHub	20+

10.4 Outils de qualité utilisés

- **JUnit 5** : Framework de tests unitaires

- **Mockito** : Framework de simulation pour les tests
- **JaCoCo** : Outil de mesure de couverture de code
- **Maven** : Outil de build et gestion des dépendances
- **Git** : Système de contrôle de version
- **GitHub** : Plateforme de collaboration et versionnement
- **PlantUML** : Outil de génération de diagrammes UML
- **GitHub Projects** : Outil de gestion de projet Scrum